



Computer Networks

EE357

Application Layer-Part 2

Haiming Jin

The slides are adapted from those provided by Prof. Romit Roy Choudhury.

Chapter 2: Application layer

- ❑ 2.1 Principles of network applications
- ❑ 2.2 Web and HTTP
- ❑ 2.3 FTP
- ❑ 2.4 Electronic Mail
 - ❖ SMTP, POP3, IMAP
- ❑ 2.5 DNS
- ❑ 2.7 Socket programming with TCP
- ❑ 2.8 Socket programming with UDP

DNS: Domain Name System

- ❑ Imagine a world without DNS
- ❑ You would have to remember the IP addresses of
 - ❖ Every website you want to visit
 - ❖ Your bookmarks will be a list of IP addresses

- ❖ You will speak like

*“I went to 167.33.24.10, and there was an awesome
link to 153.11.35.81... “*

DNS: Domain Name System

People: many identifiers:

- ❖ SSN, name, passport #

Internet hosts, routers:

- ❖ IP address (32 bit) - used for addressing datagrams
- ❖ “name”, e.g.,
www.yahoo.com - used by humans

Q: map between IP addresses and name ?

Domain Name System:

- ❑ *distributed database* implemented in hierarchy of many *name servers*
- ❑ *application-layer protocol* host, name servers to communicate to *resolve* names (address/name translation)
 - ❖ note: core Internet function, implemented as application-layer protocol
 - ❖ complexity at network’s “edge”

DNS

DNS services

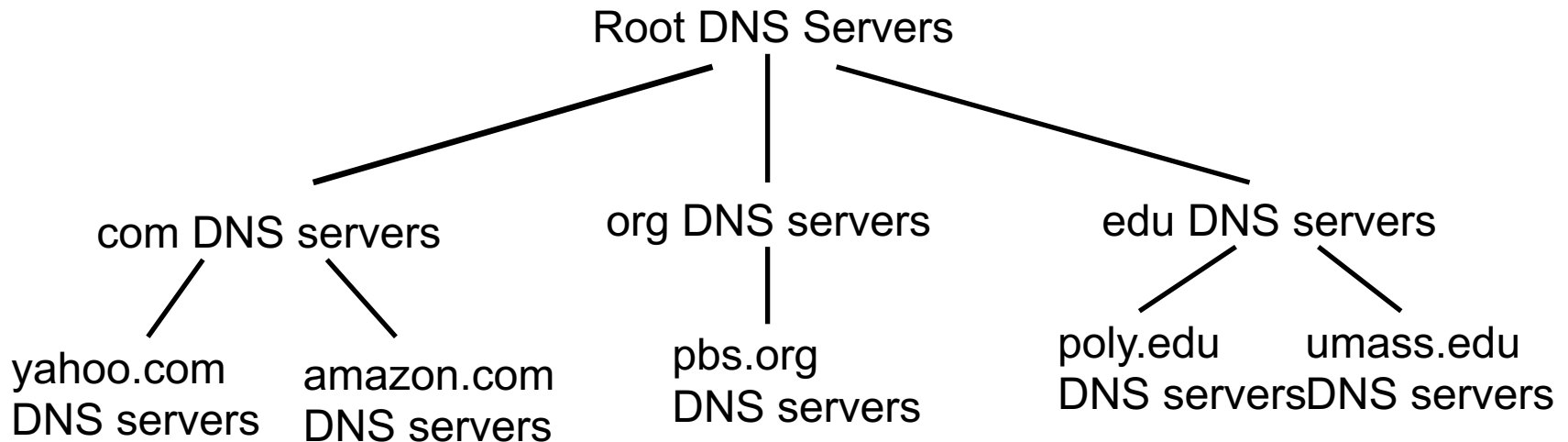
- ❑ Hostname to IP address translation
- ❑ Host aliasing
 - ❖ Canonical and alias names
- ❑ Load distribution
 - ❖ Replicated Web servers: set of IP addresses for one canonical name

Why not centralize DNS?

- ❑ single point of failure
- ❑ traffic volume
- ❑ distant centralized database

doesn't *scale*!

Distributed, Hierarchical Database

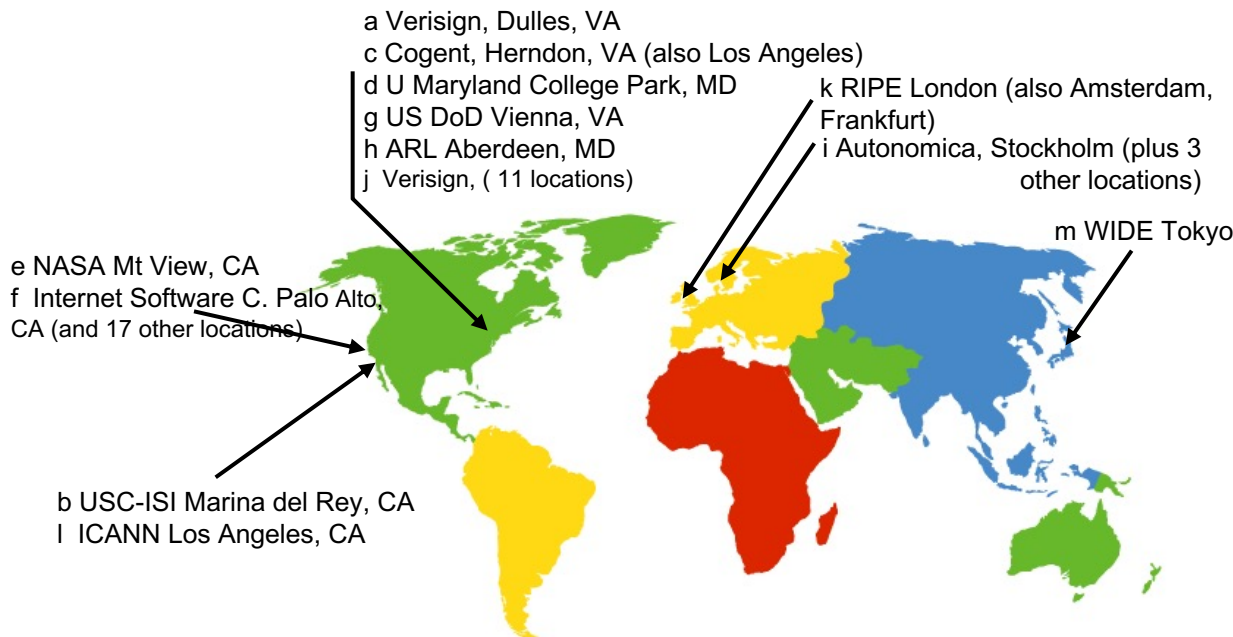


Client wants IP for www.amazon.com; 1st approx:

- ❑ Client queries a root server to find [.com](http://com) DNS server
- ❑ Client queries com DNS server to get amazon.com DNS server
- ❑ Client queries amazon.com DNS server to get IP address for www.amazon.com

DNS: Root name servers

- ❑ contacted by local name server that can not resolve name
- ❑ root name server:
 - ❖ contacts **authoritative name server** if name mapping not known
 - ❖ gets mapping
 - ❖ returns mapping to local name server



TLD and Authoritative Servers

□ Top-level domain (TLD) servers:

- ❖ responsible for com, org, net, edu, etc.
- ❖ all top-level country domains uk, fr, ca, jp.
- ❖ Educause for edu TLD

□ Authoritative DNS servers:

- ❖ An organization's DNS servers,
 - providing authoritative hostname to IP mappings for organization's servers (e.g., Web and mail).
- ❖ Can be maintained by organization or service provider

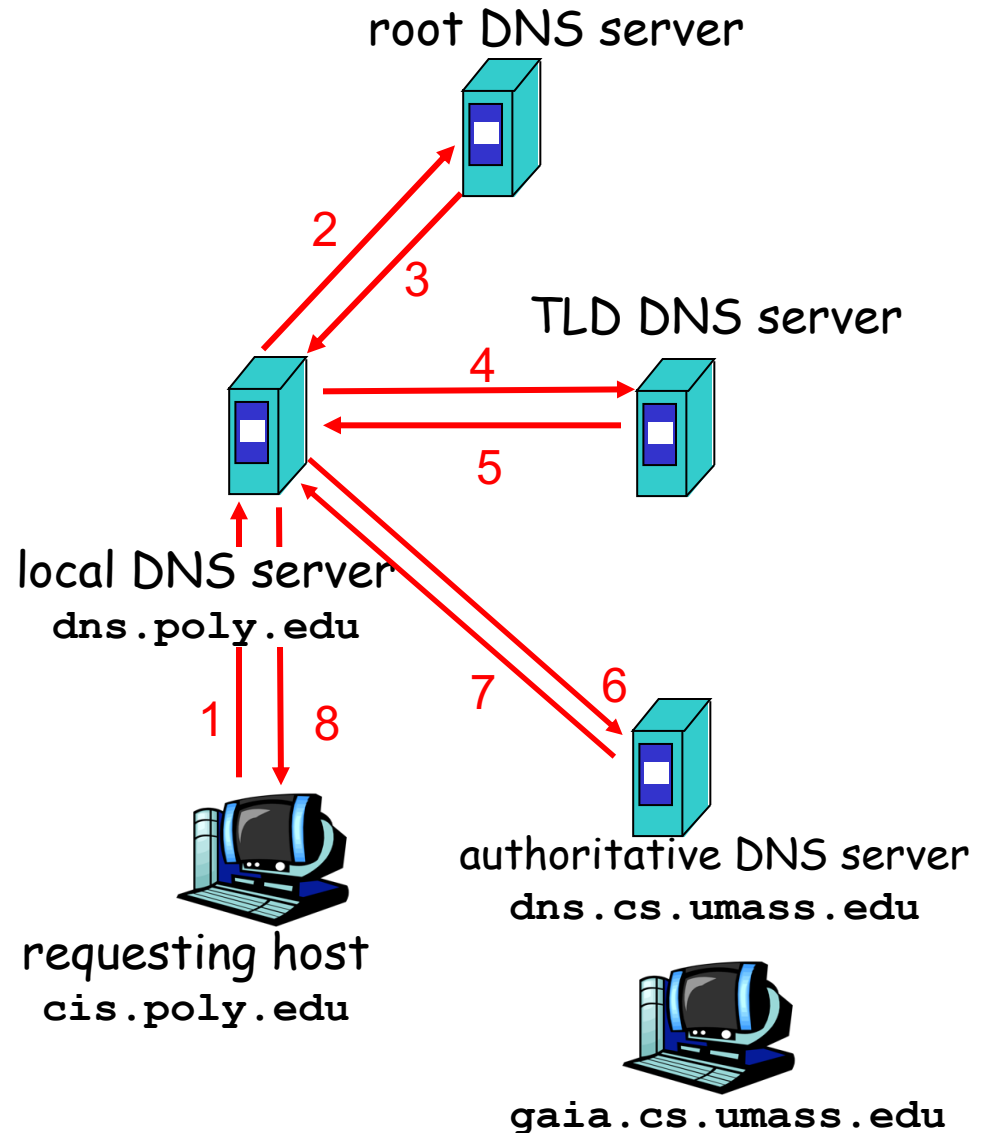
Local Name Server

- ❑ Does not strictly belong to hierarchy
- ❑ Each ISP (residential, company, univ) has one.
 - ❖ Also called “default name server”
- ❑ When a host makes a DNS query
 - ❖ query is sent to its local DNS server
 - ❖ Acts as a proxy, forwards query into hierarchy.

Example

❑ Iterative Querying

Host at cis.poly.edu
wants IP address for
gaia.cs.umass.edu



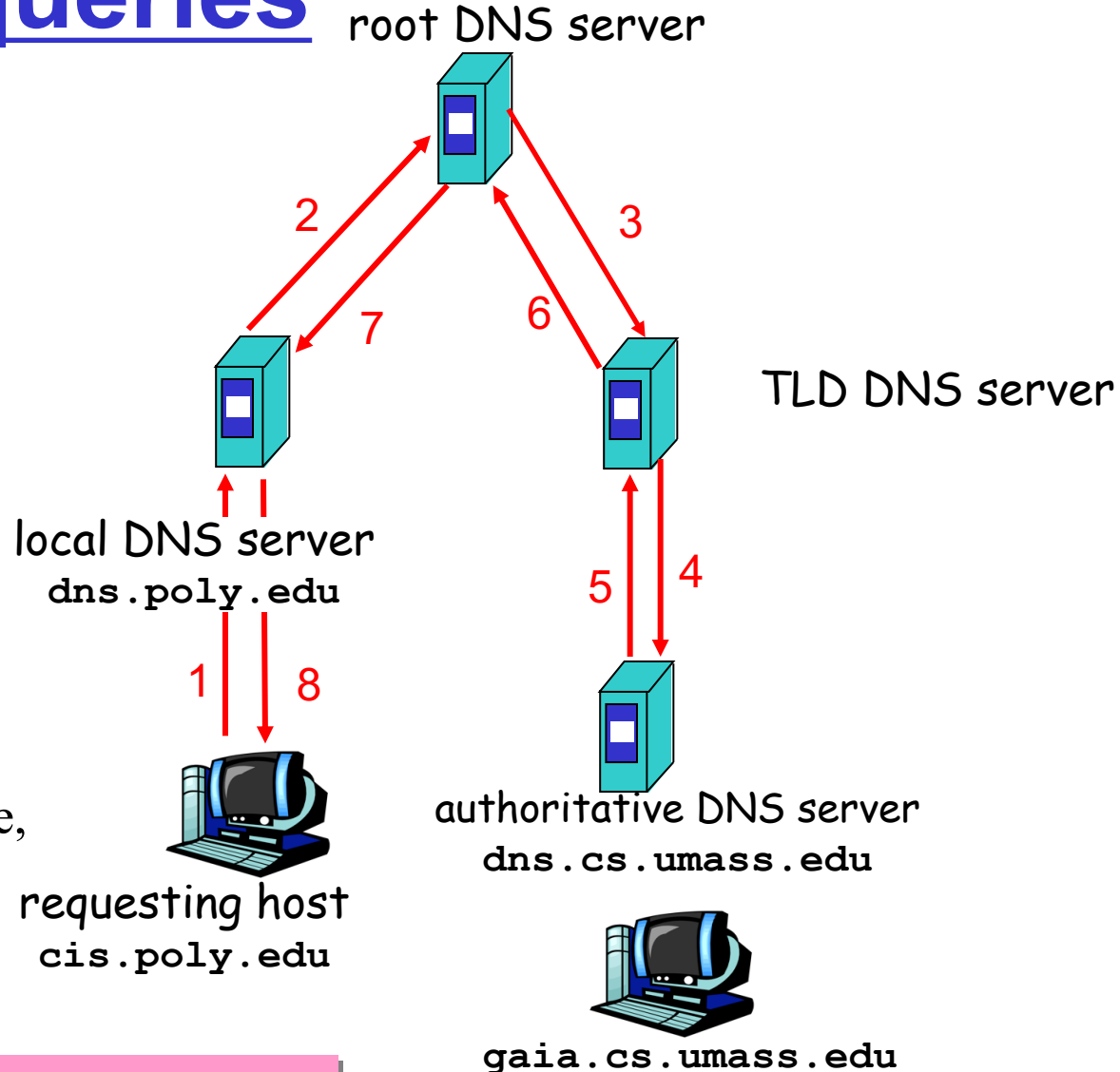
Recursive queries

recursive query:

- ❑ puts burden of name resolution on contacted name server
- ❑ heavy load?

iterated query:

- ❑ contacted server replies with name of server to contact
- ❑ “I don’t know this name, but ask this server”



Which is a better design choice?

DNS: caching

- ❑ Once (any) name server learns mapping, it *caches* mapping
 - ❖ cache entries timeout (disappear) after some time
 - ❖ TLD servers typically cached in local name servers
 - Thus root name servers not often visited

DNS records

DNS: distributed db storing resource records (RR)

RR format: (name, value, type, ttl)

□ Type=A

- ❖ **name** is hostname
- ❖ **value** is IP address

□ Type=NS

- ❖ **name** is domain (e.g. foo.com)
- ❖ **value** is hostname of authoritative name server for this domain

□ Type=CNAME

- ❖ **name** is alias name for some “canonical” (the real) name
www.ibm.com is really
servereast.backup2.ibm.com
- ❖ **value** is canonical name

□ Type=MX

- ❖ **value** is name of mailserver associated with **name**

DNS protocol, messages

DNS protocol : *query* and *reply* messages, both with same *message format*

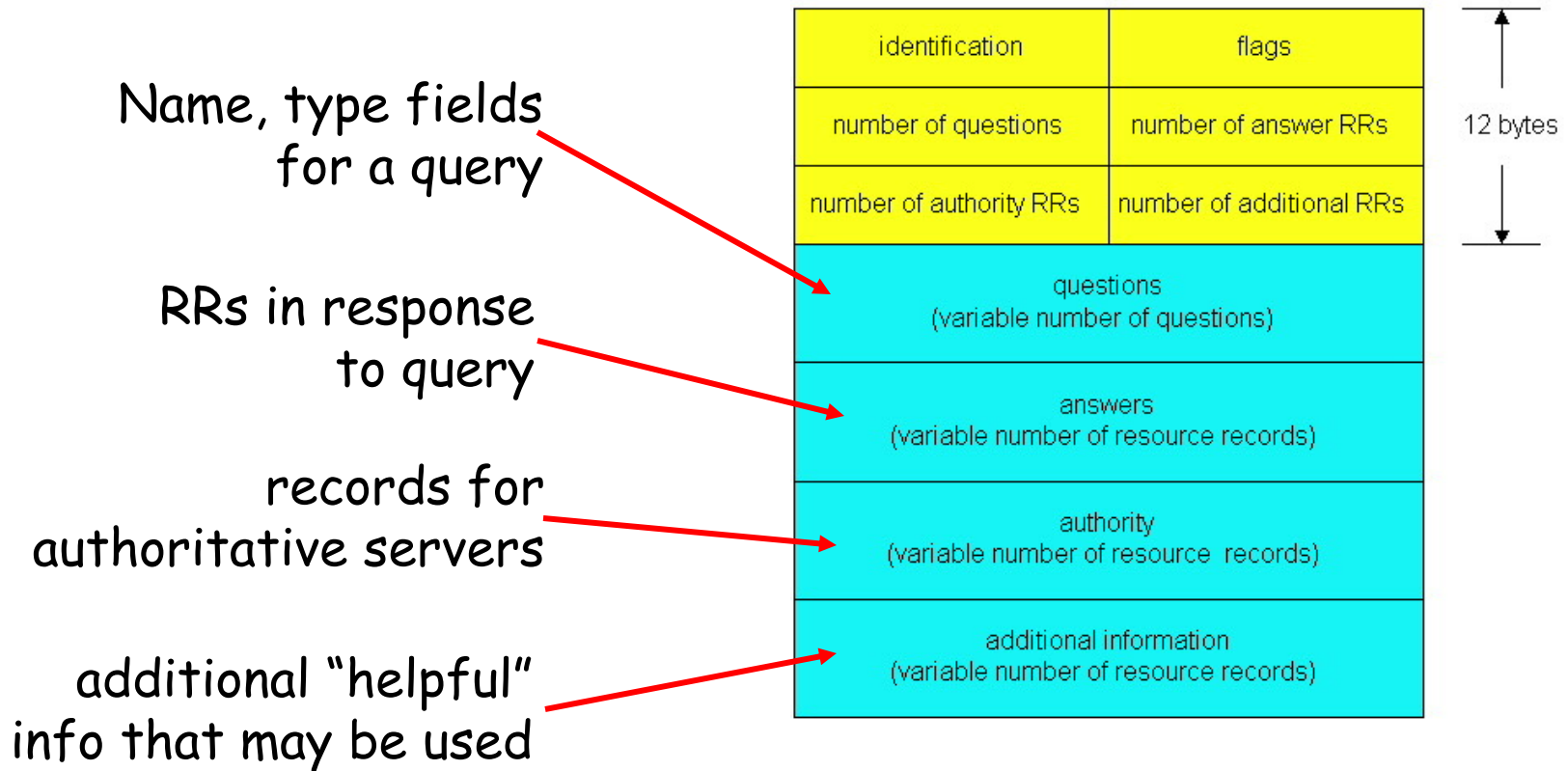
msg header

- ❑ **identification**: 16 bit # for query, reply to query uses same #
- ❑ **flags**:
 - ❖ query or reply
 - ❖ recursion desired
 - ❖ recursion available
 - ❖ reply is authoritative

identification	flags
number of questions	number of answer RRs
number of authority RRs	number of additional RRs
questions (variable number of questions)	
answers (variable number of resource records)	
authority (variable number of resource records)	
additional information (variable number of resource records)	

↑
12 bytes
↓

DNS protocol, messages



Inserting records into DNS

- ❑ Example: just created startup “Network Utopia”
- ❑ Register name networkutopia.com at a **registrar** (e.g., Network Solutions)
 - ❖ Need to provide registrar with names and IP addresses of your authoritative name server (primary and secondary)
 - ❖ Registrar inserts two RRs into the com TLD server:

```
(networkutopia.com, dns1.networkutopia.com, NS)
(dns1.networkutopia.com, 212.212.212.1, A)
```

- ❑ Put in authoritative server Type A record for www.networkutopia.com and Type MX record for networkutopia.com

Questions ?

Chapter 2: Application layer

- ❑ 2.1 Principles of network applications
- ❑ 2.2 Web and HTTP
- ❑ 2.3 FTP
- ❑ 2.4 Electronic Mail
 - ❖ SMTP, POP3, IMAP
- ❑ 2.5 DNS
- ❑ 2.7 Socket programming with TCP
- ❑ 2.8 Socket programming with UDP

Socket programming

Goal: learn how to build client/server application that communicate using sockets

Socket API

- ❑ introduced in BSD4.1 UNIX, 1981
- ❑ explicitly created, used, released by apps
- ❑ client/server paradigm
- ❑ two types of transport service via socket API:
 - ❖ unreliable datagram
 - ❖ reliable, byte stream-oriented

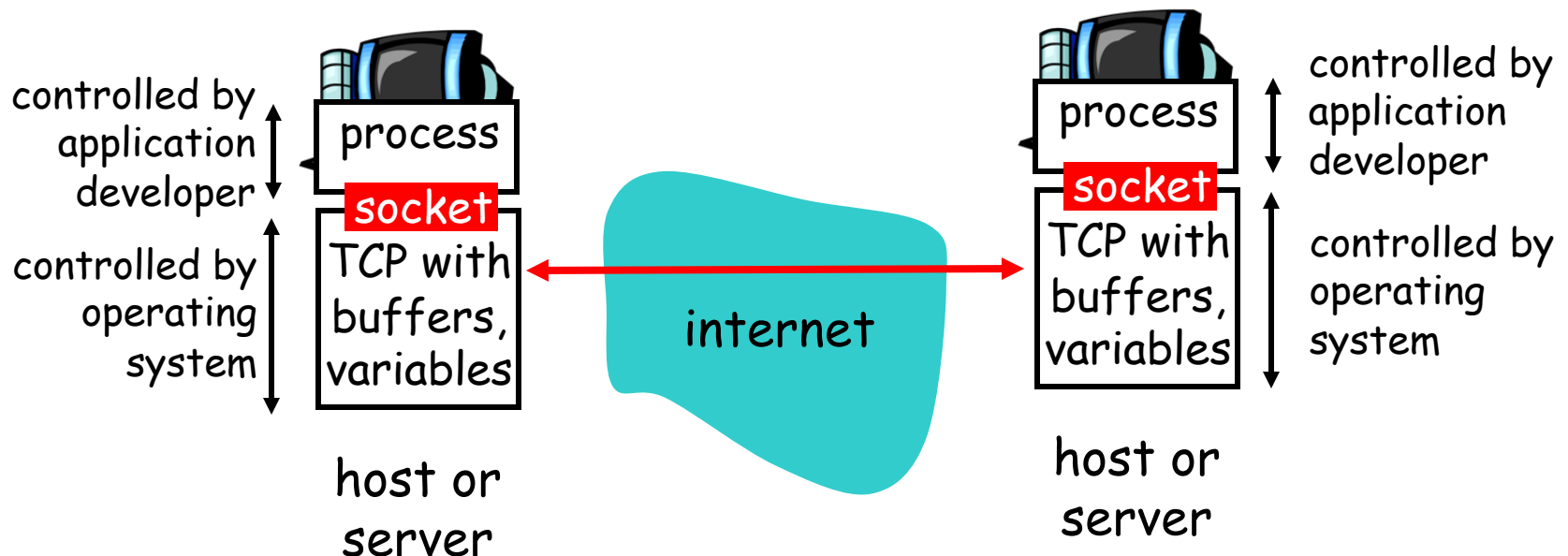
socket

a *host-local, application-created, OS-controlled* interface (a "door") into which application process can *both send and receive* messages to/from another application process

Socket-programming using TCP

Socket: a door between application process and end-end-transport protocol (UCP or TCP)

TCP service: reliable transfer of **bytes** from one process to another



Socket programming *with TCP*

Client must contact server

- ❑ server process must first be running
- ❑ server must have created socket (door) that welcomes client's contact

Client contacts server by:

- ❑ creating client-local TCP socket
- ❑ specifying IP address, port number of server process
- ❑ When **client creates socket**: client TCP establishes connection to server TCP

- ❑ When contacted by client, **server TCP creates new socket** for server process to communicate with client
 - ❖ allows server to talk with multiple clients
 - ❖ source port numbers used to distinguish clients ([more in Chap 3](#))

application viewpoint

TCP provides reliable, in-order transfer of bytes ("pipe") between client and server

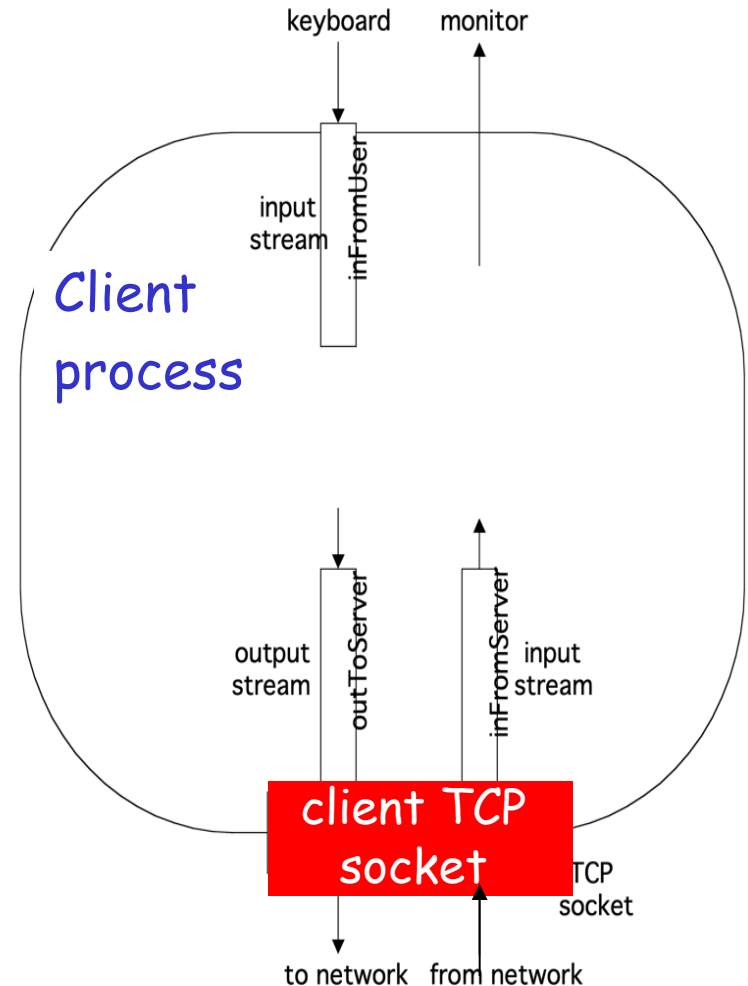
Stream jargon

- ❑ A **stream** is a sequence of characters that flow into or out of a process.
- ❑ An **input stream** is attached to some input source for the process, e.g., keyboard or socket.
- ❑ An **output stream** is attached to an output source, e.g., monitor or socket.

Socket programming with TCP

Example client-server app:

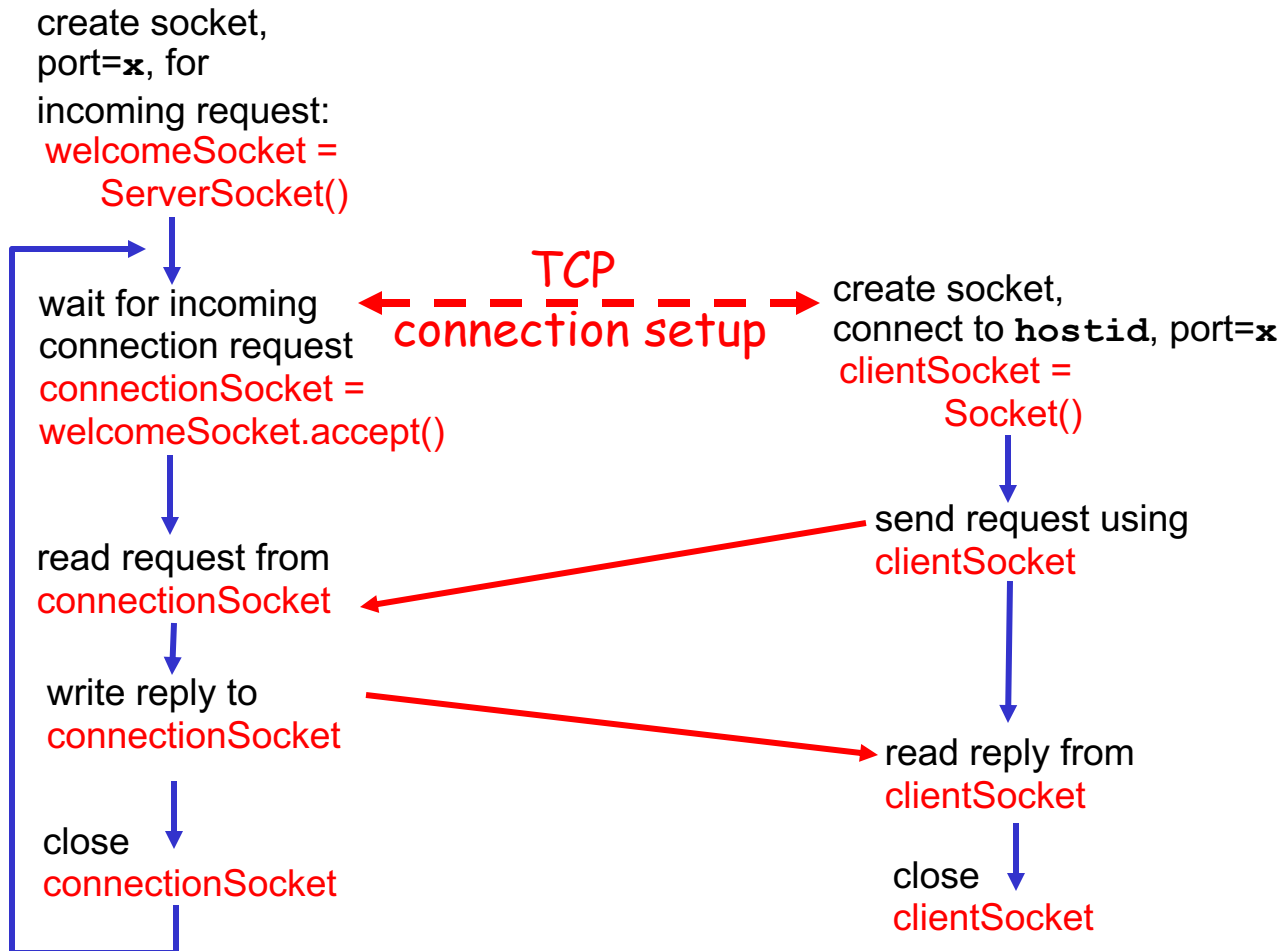
- 1) client reads line from standard input (**inFromUser** stream) , sends to server via socket (**outToServer** stream)
- 2) server reads line from socket
- 3) server converts line to uppercase, sends back to client
- 4) client reads, prints modified line from socket (**inFromServer** stream)



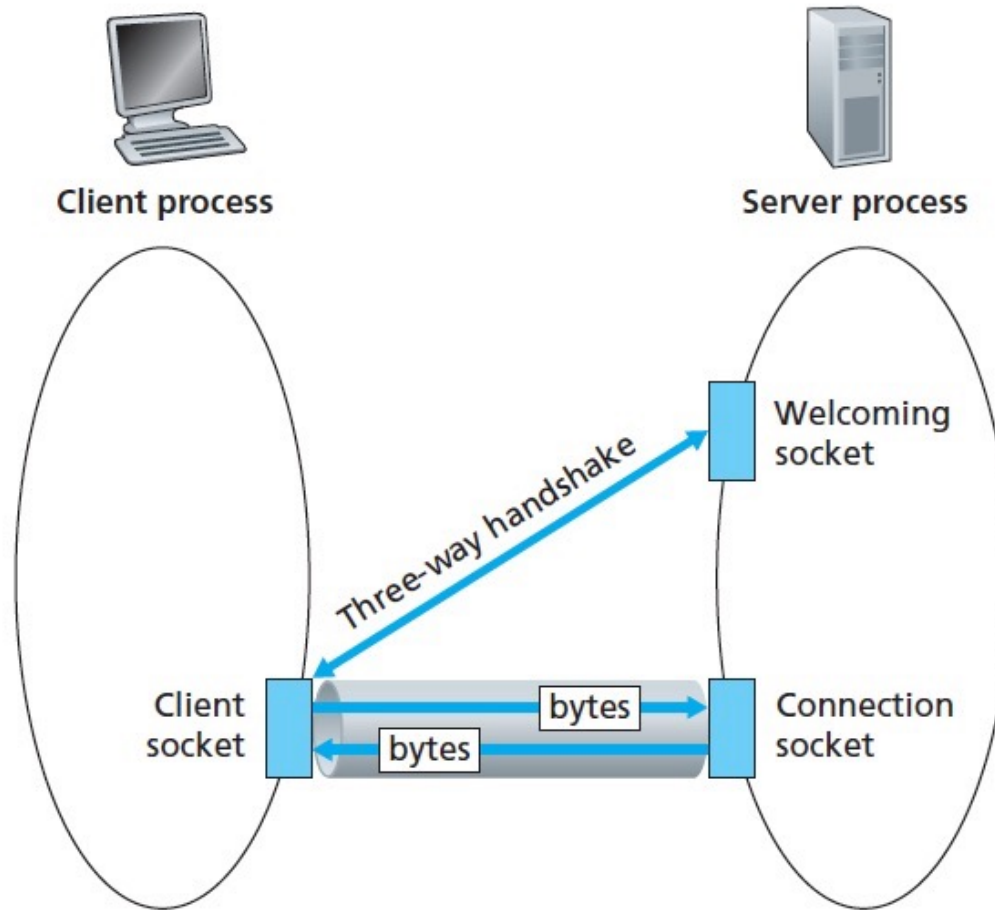
Client/server socket interaction: TCP

Server (running on `hostid`)

Client



Client/server socket interaction: TCP



Example: Java client (TCP)

```
import java.io.*;
import java.net.*;
class TCPClient {
```

```
    public static void main(String argv[]) throws Exception
    {
```

```
        String sentence;
        String modifiedSentence;
```

Create
input stream



```
        BufferedReader inFromUser =
            new BufferedReader(new InputStreamReader(System.in));
```

Create
client socket,
connect to server



```
        Socket clientSocket = new Socket("hostname", 6789);
```

Create
output stream
attached to socket



```
        DataOutputStream outToServer =
            new DataOutputStream(clientSocket.getOutputStream());
```

Example: Java client (TCP), cont.

Create
input stream
attached to socket

→

```
BufferedReader inFromServer =  
    new BufferedReader(new  
        InputStreamReader(clientSocket.getInputStream()));  
  
sentence = inFromUser.readLine();  
  
Send line  
to server

→



```
outToServer.writeBytes(sentence + '\n');
```



Read line  
from server



→



```
modifiedSentence = inFromServer.readLine();

System.out.println("FROM SERVER: " + modifiedSentence);

clientSocket.close();

}
}
```


```

Example: Java server (TCP)

```
import java.io.*;  
import java.net.*;
```

```
class TCPServer {
```

```
    public static void main(String argv[]) throws Exception  
    {
```

```
        String clientSentence;  
        String capitalizedSentence;
```

Create
welcoming socket
at port 6789

```
        ServerSocket welcomeSocket = new ServerSocket(6789);
```

Wait, on welcoming
socket for contact
by client

```
        while(true) {
```

```
            Socket connectionSocket = welcomeSocket.accept();
```

Create input
stream, attached
to socket

```
            BufferedReader inFromClient =  
                new BufferedReader(new  
                    InputStreamReader(connectionSocket.getInputStream()));
```

Example: Java server (TCP), cont

Create output
stream, attached
to socket

```
DataOutputStream outToClient =  
    new DataOutputStream(connectionSocket.getOutputStream());
```

Read in line
from socket

```
clientSentence = inFromClient.readLine();
```

```
capitalizedSentence = clientSentence.toUpperCase() + '\n';
```

Write out line
to socket

```
outToClient.writeBytes(capitalizedSentence);
```

```
}  
}  
}
```

End of while loop,
loop back and wait for
another client connection

Chapter 2: Application layer

- ❑ 2.1 Principles of network applications
- ❑ 2.2 Web and HTTP
- ❑ 2.3 FTP
- ❑ 2.4 Electronic Mail
 - ❖ SMTP, POP3, IMAP
- ❑ 2.5 DNS
- ❑ 2.7 Socket programming with TCP
- ❑ 2.8 Socket programming with UDP

Socket programming *with UDP*

UDP: no “connection” between client and server

- ❑ no handshaking
- ❑ sender explicitly attaches IP address and port of destination to each packet
- ❑ server must extract IP address, port of sender from received packet

UDP: transmitted data may be received out of order, or lost

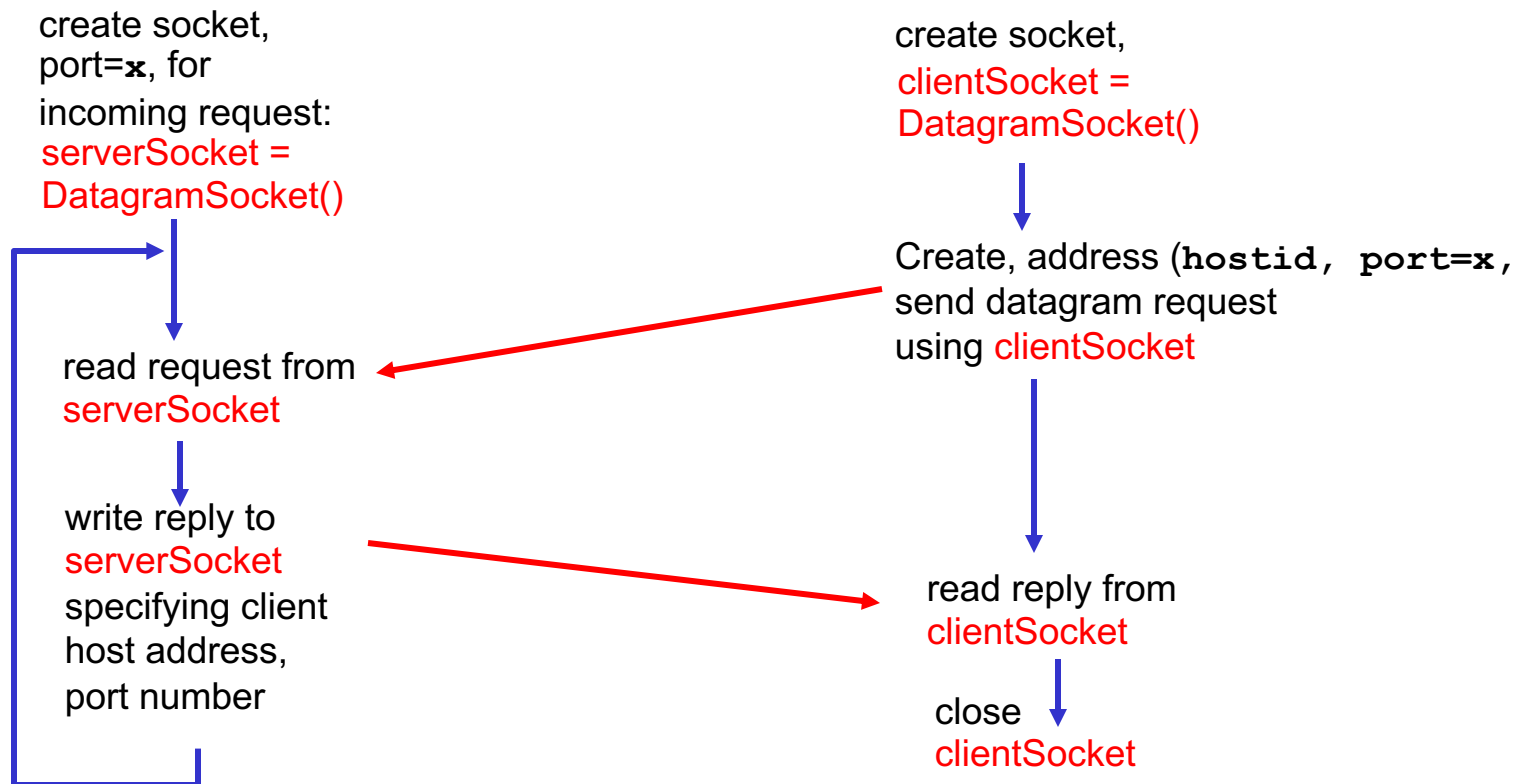
application viewpoint

UDP provides unreliable transfer of groups of bytes (“datagrams”) between client and server

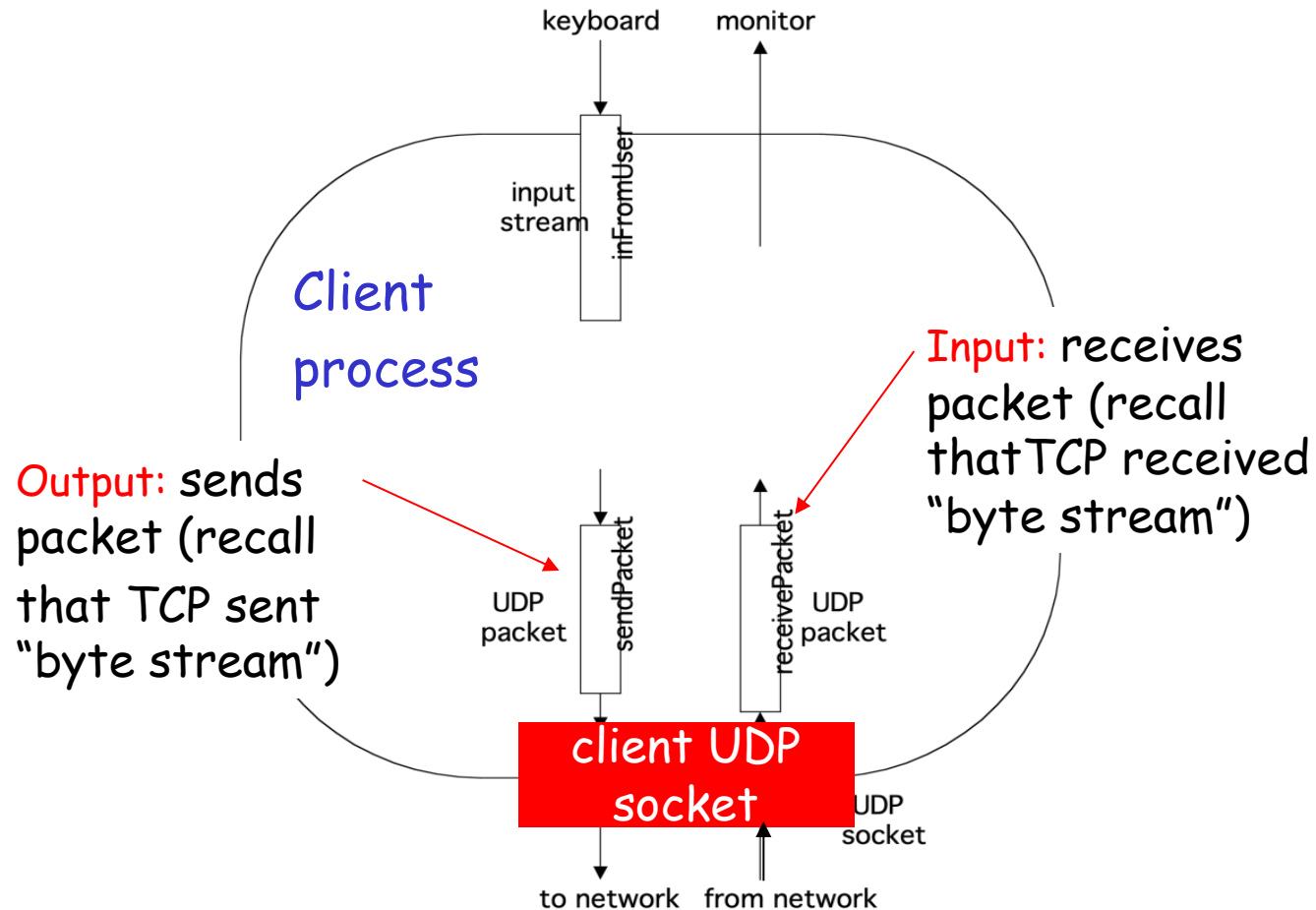
Client/server socket interaction: UDP

Server (running on `hostid`)

Client



Example: Java client (UDP)



Example: Java client (UDP)

```
import java.io.*;  
import java.net.*;
```

```
class UDPClient {  
    public static void main(String args[]) throws Exception  
    {
```

Create
input stream

```
        BufferedReader inFromUser =  
            new BufferedReader(new InputStreamReader(System.in));
```

Create
client socket

```
        DatagramSocket clientSocket = new DatagramSocket();
```

Translate
hostname to IP
address using DNS

```
        InetAddress IPAddress = InetAddress.getByName("hostname");
```

```
        byte[] sendData = new byte[1024];
```

```
        byte[] receiveData = new byte[1024];
```

```
        String sentence = inFromUser.readLine();
```

```
        sendData = sentence.getBytes();
```

Example: Java client (UDP), cont.

Create datagram
with data-to-send,
length, IP addr, port

Send datagram
to server

Read datagram
from server

```
DatagramPacket sendPacket =  
    new DatagramPacket(sendData, sendData.length, IPAddress, 9876);  
  
clientSocket.send(sendPacket);  
  
DatagramPacket receivePacket =  
    new DatagramPacket(receiveData, receiveData.length);  
  
clientSocket.receive(receivePacket);  
  
String modifiedSentence =  
    new String(receivePacket.getData());  
  
System.out.println("FROM SERVER:" + modifiedSentence);  
clientSocket.close();  
}  
}
```

Example: Java server (UDP)

```
import java.io.*;  
import java.net.*;
```

```
class UDPServer {  
    public static void main(String args[]) throws Exception  
    {
```

Create
datagram socket
at port 9876



```
        DatagramSocket serverSocket = new DatagramSocket(9876);
```

```
        byte[] receiveData = new byte[1024];  
        byte[] sendData = new byte[1024];
```

```
        while(true)  
        {
```

Create space for
received datagram



```
            DatagramPacket receivePacket =  
                new DatagramPacket(receiveData, receiveData.length);
```

Receive
datagram



```
            serverSocket.receive(receivePacket);
```

Example: Java server (UDP), cont

```
String sentence = new String(receivePacket.getData());
```

Get IP addr
port #, of
sender

```
InetAddress IPAddress = receivePacket.getAddress();
```

```
int port = receivePacket.getPort();
```

Which address is this?

```
String capitalizedSentence = sentence.toUpperCase();
```

```
sendData = capitalizedSentence.getBytes();
```

Create datagram
to send to client

```
DatagramPacket sendPacket =  
    new DatagramPacket(sendData, sendData.length, IPAddress,  
                        port);
```

Write out
datagram
to socket

```
serverSocket.send(sendPacket);
```

```
}  
}  
}
```

End of while loop,
loop back and wait for
another datagram

Sockets and Ports

What is the difference between sockets and ports?

Sockets are physical telephones

Ports are extension numbers
IP address is the phone number

Chapter 2: Summary

Our study of network apps now complete!

- Application architectures

- ❖ client-server
- ❖ P2P
- ❖ hybrid

- application service requirements:

- ❖ reliability, bandwidth, delay

- Internet transport service model

- ❖ connection-oriented, reliable: TCP
- ❖ unreliable, datagrams: UDP

- specific protocols:

- ❖ HTTP
- ❖ FTP
- ❖ SMTP, POP, IMAP
- ❖ DNS

- socket programming

Chapter 2: Summary

Most importantly: learned about *protocols*

- typical request/reply message exchange:
 - ❖ client requests info or service
 - ❖ server responds with data, status code
- message formats:
 - ❖ headers: fields giving info about data
 - ❖ data: info being communicated
- control vs. data msgs
 - ❖ in-band, out-of-band
- centralized vs. decentralized
- stateless vs. stateful
- reliable vs. unreliable msg transfer
- “complexity at network edge”

Questions?